

Fabuloso - Phase 2 Documentation

Student Number: s5353249

Student Name: Eden Moss

Course: 3813ICT Full Stack Development

Application: Fabuloso / Fabulari

Repository: <https://github.com/R4nd0mHUMAN/Fabulari>

1. Project specification

Fabuloso is a responsive multi-user group-chat application. Angular provides the browser UI; Node.js/Express provides the REST API; the official `mongodb` Node.js driver provides persistent MongoDB access; `bcrypt` hashes passwords; `Multer` accepts profile images; and `Socket.io` provides live channel communication and online/offline presence. Phase 2 does **not** use `Mongoose`. MongoDB database `fabuloso_phase2` is the persistent source of truth. `server/data/mongodb-export.json` is a read-only inspection snapshot and is never loaded as application state.

Functional requirements implemented

- One-time Super Admin bootstrap, normal registration/login/logout, route guards, server-side role checks, private profiles, passwordchange and account deletion.
- Group creation requests, Super Admin approval/rejection, group membership requests, age restrictions, group-adminpromotion/demotion, last-admin protection, member removal, group editing and channel CRUD.
- Real-time text and PNG/JPEG/GIF chat, join/leave notices, synchronized own-message deletion, online/offline indicators and fivepersistent message slots per channel. During one open chat session Angular may display more than five live messages; re-entry loads the newest five persisted slots.
- Profile image upload with profile pictures displayed on the profile/home interfaces and beside the sender in chat.
- Chat image upload with client and server validation and a 2 MB decoded-image limit. `Socket.io`'s transport buffer is configured above theapplication limit so a valid image close to 2 MB can still be transmitted after Base64 expansion.
- Super Admin user/banned-user/audit interfaces and audit filtering.
- Responsive user and Super Admin visual themes plus supplied Fabulari branding.

2. Architecture and repository layout

Browser

- > Angular standalone components / Router / HttpClient
- > REST requests to Express :3000
- > native `mongodb` Node.js driver
- > MongoDB `fabuloso_phase2`

Browser ChatRoom <-> Socket.io <-> Node server <-> MongoDB messages

|
-> online-user presence state

Assignment

<code>src/app/</code>	Angular components, route guards, templates and tests
<code>public/assets/Images/</code>	supplied/design imagery including Fabulari logo
<code>e2e/</code>	Playwright end-to-end tests
<code>server/</code>	
<code>routes/</code>	Express REST route modules
<code>middleware/</code>	server authentication/role middleware
<code>tests/</code>	Node backend unit/validation tests
<code>scripts/inspect.js</code>	direct native-driver MongoDB inspection
<code>database.js</code>	native MongoClient data-access layer
<code>server.js</code>	Express + Socket.io entry point
<code>export-snapshot.js</code>	optional JSON inspection export
<code>uploads/</code>	uploaded profile-picture files
<code>Phase2.md</code>	this documentation
<code>Phase2_Submission.pdf</code>	submission copy of this documentation

Version-control approach

Development should be committed in small logical units rather than one final dump. Appropriate commit subjects include `feat:`
`add`

group membership approval, fix: preserve five MongoDB chat slots, feat: add socket image messages, test: add backend validation tests, and docs: complete Phase 2 API documentation.

Generated dependencies

(node_modules) are excluded. The repository root separates Angular, server, tests, assets and documentation so a marker can locate each concern quickly. The tutor must be added as a collaborator if the repository is private.

3. Persistent data structures

Collection	Important fields	Purpose
users	id, username, email, DOB/age, bcrypt password, role, groups, profilePicture	account/authentication state
groups	id, name, description, ageLimit, theme, members[], admins[]	group membership and group-specific authority
channels	id, groupId, name, description	rooms belonging to groups
requests	type, requester, group, status, reason, timestamps	group-create and join workflow queue
messages	channelId, userId, username, profilePicture, text, image, deleted, timestamp	newest five persistent chat slots per channel
audit_logs	actor, action, details, timestamp	trace of administrative mutations
banned_emails	email, deletedAt	prevents deleted/banned addresses being reused

Application UUIDs are stored in `id`; MongoDB `_id` remains an internal database identifier. The custom native-driver document layer deliberately leaves MongoDB's immutable `_id` untouched when updating records. Indexes are created for UUID/email lookup and channel-message history. Password hashes are removed from API responses, terminal inspection output and JSON inspection exports.

Uploaded profile-picture binaries are stored under `server/uploads/`; the user's `profilePicture` field stores the `/uploads/...` path. Express exposes this directory as static content while MongoDB persists the relationship between the user and uploaded image.

Online/offline presence is intentionally transient rather than stored in MongoDB. `server.js` keeps a count of active Socket.io connections per user so a user with multiple active sockets is not incorrectly marked offline when only one connection closes.

4. Angular architecture

The application uses standalone Angular components. `Login`, `Register` and `Bootstrap` handle entry. `UserHome`, `GroupSearch`, `CreateGroup`, `GroupRequests`, `MyGroups`, `GroupAdmin`, `Profile` and `ChatRoom` implement the user workflow. `SuperAdmin`, `UserManagement`, `BannedUsers`, `GroupRequests` and `AuditLogs` implement privileged workflows. `authGuard`, `guestGuard` and `superAdminGuard` protect client routes; these are usability controls only, so Express repeats authorization checks server-side.

`HttpClient` performs REST communication. `socket.io-client` maintains the live channel connection. Local storage retains the current application user identity for this teaching implementation. `ChangeDetectorRef` is used where asynchronous HTTP/Socket.io callbacks need an explicit redraw, including administration lists, chat messages and presence updates.

`ChatRoom` separates persistent history from current-session history. It receives the five persisted messages on entry, then appends all new live messages during that visit. It renders each sender's profile picture, current online/offline indicator, timestamp, text/image content and own-message delete control. The chat history scrolls independently so the channel header and composer remain available.

The UI uses responsive CSS, contained scrolling for chat, clear labels/buttons, alt text on branded/shared/profile images, and separate user/Super Admin themes.

5. REST API documentation

All protected requests carry `x-user-id`. The server re-fetches that user and enforces authority. Error responses use `{ "message": "human readable explanation" }` with appropriate HTTP status codes.

Authentication

Method / route	Parameters/body	Success return / purpose
GET <code>/api/auth/bootstrap-status</code>	none	<code>{bootstrapRequired}</code> ; checks whether initial Super Admin is needed
POST <code>/api/auth/bootstrap</code>	username/email/password/name/DOB	201; creates the single bcrypt-hashed Super Admin
POST <code>/api/auth/register</code>	username/email/password/name/DOB	201 <code>{message, user}</code> ; rejects duplicate/banned email and invalid DOB
POST <code>/api/auth/login</code>	username or email + password	safe user object; bcrypt verification
GET <code>/api/auth/me</code>	authenticated header	current safe user
DELETE <code>/api/auth/me</code>	authenticated header	deletes account if last-admin rule permits; bans email
POST <code>/api/auth/logout</code>	none	logout acknowledgement; Angular clears local state

Groups, membership and requests

Method / route	Parameters/body	Success return / purpose
GET <code>/api/groups</code>	none	alphabetical groups visible for discovery
GET <code>/api/groups/:id</code>	group UUID	one group
POST <code>/api/groups/requests</code>	name, description, ageLimit	pending group-create request
GET <code>/api/groups/requests/user/:id</code>	user UUID	user's group-create requests
GET <code>/api/groups/membership-requests/user/:id</code>	user UUID	user's join-request history
GET <code>/api/groups/requests/creation</code>	Super Admin	pending creation queue
Method / route	Parameters/body	Success return / purpose
POST <code>/api/groups/requests/:id/approve</code>	request UUID	creates group; requester becomes member + first admin
POST <code>/api/groups/requests/:id/reject</code>	request UUID, reason	rejected request
POST <code>/api/groups/:id/join-request</code>	group UUID	age-checks then creates pending join request
GET <code>/api/groups/requests/admin/:id</code>	admin UUID	pending joins for administered groups
POST <code>/api/groups/:gid/membership-requests/:r id/approve</code>	group/request UUID	adds requester as member
POST <code>/api/groups/:gid/membership-requests/:r id/reject</code>	IDs + reason	rejects join request
GET <code>/api/groups/user/:id</code>	user UUID	subscribed groups plus member/admin role
GET <code>/api/groups/owned/:id</code>	user UUID	groups administered by user
GET <code>/api/groups/:id/members</code>	group UUID	alphabetical member list
PUT <code>/api/groups/:id</code>	name/description/ageLimit/theme	validated group update; removes newly underage members
POST <code>/api/groups/:id/admins/:uid/promote</code>	group/user UUID	promotes an existing member

POST /api/groups/:id/admins/:uid/demote	group/user UUID	demotes while preserving membership; refuses last admin
DELETE /api/groups/:id/members/:uid	group/user UUID	removes non-admin member

Channels, profiles and Super Admin

Method / route	Parameters/body	Success return / purpose
POST /api/groups/channels	groupId, name, description	Group Admin creates channel
GET /api/groups/:id/rooms	group UUID	member-only channel list
GET /api/groups/channels/group/:id	group UUID	channel administration list
PUT /api/groups/channels/:id	name/description	Group Admin edits channel
DELETE /api/groups/channels/:id	channel UUID	Group Admin deletes channel
GET /api/profile/:id	owner UUID	private safe profile
PUT /api/profile/:id	editable name fields	updates profile; email remains immutable
PUT /api/profile/:id/password	currentPassword/newPassword	verifies old password then re-hashes
PUT /api/profile/:id/picture	multipart profilePicture	PNG/JPEG/GIF max 2 MB; returns /uploads/... path
GET /api/admin/users	Super Admin	alphabetical safe user list
GET /api/admin/banned-emails	Super Admin	read-only list of deleted/banned addresses
DELETE /api/admin/users/:id	Super Admin + user UUID	permanent deletion/ban with admin-successor check
GET /api/admin/audit?type=&date=	optional action/date filters	newest-first audit records

6. Socket.io protocol

Event	Direction	Payload	Behaviour
join-channel	client -> server	channelId, userId, username	verifies group membership, stores trusted username/profile picture, joins Socket.io room
chat-history	server -> client	newest five message documents	initial/re-entry persistent history including sender avatar path
send-message	client -> server	channelId, userId, username, text, optional image data URL	validates socket identity and image type/2 MB size; persists trusted sender data
new-message	server -> room	saved message	real-time text/image/avatar broadcast to all active viewers
delete-message	client -> server	channelId, messageId, userId	owner-only deletion request
message-deleted	server -> room	messageId	synchronizes blank/deleted slot in active clients
system-message	server -> room	text	live join/leave notice
online-users	server -> room	array of online user IDs	updates green/grey presence indicators
chat-error	server -> client	message	user-friendly rejected socket operation

The server does not trust the username supplied by the browser during `join-channel`; it retrieves the user from MongoDB and stores the trusted username and profile-picture path on `socket.data`. New messages therefore use server-known sender information.

The server retains only five MongoDB message slots per channel. The open Angular component intentionally keeps additional messages received during that visit; after leaving/re-entering it receives only the five persistent slots.

Presence uses a connection counter rather than a simple Boolean so multiple active sockets belonging to one user do not incorrectly mark that user offline when only one connection closes.

Socket.io `maxHttpBufferSize` is configured at the server level to allow the Base64 transport representation of a valid image near 2 MB. Application validation still rejects decoded images above 2 MB and accepts only PNG, JPEG and GIF.

7. Validation, security and error recovery

Validation occurs on both sides where practical. Angular prevents empty submissions and validates image selection; Express/Socket.io repeat important checks because browser controls can be bypassed. The server checks authentication, Super Admin/group-admin authority, membership, age limits, last-admin invariants, duplicate/banned emails, image MIME/size and message ownership. Passwords use bcrypt hashes. Safe-user helpers prevent hashes from leaving the server. Profile uploads use an allow-list and 2 MB Multipart limit. Chat images are allow-listed and decoded-size checked again by the socket server. MongoDB is indexed for common lookups and only the five required chat slots are retained, limiting unbounded persistent message growth.

Permanent account deletion records the email in `banned_emails`, preventing that address from being reused. Administrative mutations are recorded in `audit_logs`, and the Super Admin interface supports action/date filtering.

For this teaching build `x-user-id` is the session identity mechanism rather than a production cookie/token session. Security-critical permissions are nevertheless rechecked against MongoDB on every protected REST mutation and against channel membership when a socket joins. A production deployment would replace this identity header with signed HttpOnly session cookies/JWT transport and HTTPS.

8. Testing

Automated testing supplied

- **Angular unit/component tests:** `src/app/**/*.spec.ts`, run with `npm test`.
- **Node backend tests:** `server/tests/*.test.js`, run with `cd server && npm test`. These cover DOB/public-user safety and chat-image validation boundaries.
- **Playwright E2E:** `e2e/public-navigation.spec.ts`, run with `npm run test:e2e`. It verifies public branding and protected-route redirection. Install browser once with `npx playwright install chromium`.
- **Manual multi-browser acceptance:** multiple logged-in browser sessions were used to verify the complete user, Group Admin, Super Admin and Socket.io workflows.

Manual acceptance verified

- User-management lists load correctly; new users appear; permanent deletion removes the user and adds the email to Banned Users.
- Permanently deleted/banned emails cannot be registered again and receive friendly feedback.
- Audit records persist and action, date and combined filters work.
- Group member listing, promotion, demotion-with-membership-retained and last-admin protection work.
- Channel create, inline rename/cancel and delete update the UI and create the expected audit records.
- Join requests can be submitted, approved or rejected. Approval grants group/channel access; rejection leaves the requester outside the group; duplicate pending requests are prevented and success/error feedback is human-readable.
- Live Socket.io messages appear immediately in multiple browsers.
- More than five messages remain visible during the current chat visit, while leaving/re-entering reloads only the newest five persisted messages.
- Own-message deletion synchronizes between connected browsers.
- PNG, JPEG and GIF chat images work. Unsupported files and images above 2 MB are rejected with friendly feedback, while a valid image close to the 2 MB boundary is accepted.

- Profile pictures persist and display on profile/home pages and beside chat messages; users without an uploaded image receive the default avatar.
- Online/offline indicators update from live Socket.io presence state.
- After shutting down and restarting the application, MongoDB still retains users, groups, memberships, channels, profile-picture paths and the latest five chat messages.

Recommended Run

```
# Terminal 1 - MongoDB service running
# Terminal 2
cd server
npm install
npm test
npm run inspect
npm start

# Terminal 3 (project root)
npm install
npm test
npm run build
npm start

# Optional E2E once Chromium is installed
npm run test:e2e

# Direct database demonstration
cd server
npm run inspect
```

The direct inspection command uses the official `mongodb` driver and displays the seven collections while hiding `bcrypt` hashes.

9. Revised storyboard / UX flow

```
First run -> Bootstrap Super Admin -> Login
Normal user -> Register/Login -> User Home -> Discover/Request Group -> Approval -> My Groups
               -> Channel -> Live Chat -> Profile -> edit details/password/profile image
Group Admin -> Group Admin -> requests/members/channels/settings
Super Admin -> Dashboard -> group requests / users / banned users / audit logs
```

The Phase 1 visual concept was retained but revised for role-aware navigation, responsive framing, contained chat scrolling, consistent Fabulari branding, distinct user/Super Admin themes and longer datasets. Lists and chat panels scroll inside their frames rather than pushing critical actions off-screen.

10. Creativity, complexity and design decisions

Complexity comes from combining group-scoped authorization, asynchronous approval queues, server-enforced business invariants, native MongoDB persistence, real-time Socket.io rooms, live text/image messaging, sender avatars, connection-aware presence, synchronized deletion, two levels of administration, auditability and responsive role-specific interfaces.

Efficiency is addressed through MongoDB indexes, bounded persistent chat history, room-scoped Socket.io broadcasts and a connection-count map for presence. The five-message rule is enforced on persistent MongoDB history without unnecessarily discarding additional messages already received by an active Angular session.

Security is addressed by server-side authorization and bcrypt rather than trusting Angular visibility. Socket message identity is tied to the verified user/channel stored on the socket after joining, and message deletion checks ownership.

UX is addressed through responsive themed pages, clear navigation, contained scrolling, immediate socket updates, profile pictures, online/offline status, image previews, inline channel renaming, consistent branding and human-readable validation/errors.

A custom native MongoDB data-access layer preserves a small document-style interface for the existing route code while using the assignment-permitted official `mongodb` driver underneath. This reduced unnecessary route rewrites during migration away from Mongoose while keeping MongoDB `_id` managed by MongoDB itself.

11. Submission checklist

- Submit **one PDF** containing student name/number and repository link.
- Keep this exact documentation in the repository as `Phase2.md`.
- Ensure tutor/teaching member can access the GitHub repository if private.
- Preserve meaningful Git history; do not replace the repository with a single final commit.
- Before demonstration run `npm install` in root and `server`, start MongoDB, run automated tests, and complete the multi-browser acceptance sequence.